

Medical Cost Personal Dataset

Le Medical Cost Personal Dataset est un jeu de données utilisé pour prédire les frais médicaux annuels d'individus à partir de leurs caractéristiques personnelles et comportementales.

- Chaque ligne du dataset correspond à une personne et contient des informations telles que l'âge, le sexe, l'indice de masse corporelle (IMC), le nombre d'enfants à charge, le statut de fumeur et la région géographique.
- **La variable cible** est charges, qui représente le montant annuel des frais médicaux.

Importer les bibliothèques

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

Q1. Importer le dataset insurance.csv (1pt)

```
In [6]: df = pd.read_csv('insurance.csv')
```

Analyse exploratoire des données

Q2: À quoi sert l'analyse exploratoire des données (EDA) ? (1pt)

sert a comprendre les caracteristiques principales du jeu de donnees souvent a laide de méthodes de visualisation

Q3: Afficher les trois premières lignes du dataset(0.5pt)

```
In [7]: df.head(3)
```

```
Out[7]:
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.90	0	yes	southwest	16884.9240
1	18	male	33.77	1	no	southeast	1725.5523
2	28	male	33.00	3	no	southeast	4449.4620

Q4: Exécuter la commande suivante et préciser ce que représente chaque chiffre (0.5pt)

```
In [10]: df.shape
```

```
Out[10]: (1338, 7)
```

In []: 1338 epresente nombre de lignes dans les donnees 7: represente nbr des colonn

Q5: Afficher les informations de chaque colonne du dataset (0.5pt)

```
In [9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   charges     1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

Q6: D'après le retour de la question précédente, existe-t-il des valeurs manquantes dans le dataset ? Si oui, quel traitement doit-on effectuer ? (1.5pt)

en observant le redtour de la q5 si le nombre de valeur non nul !=0 et =1338 il n ya pas des valeurs manquantes NAN s il existe des valeurs mnquantes NAN alors on faire imputation ou suppression

Q7: Vérifiez si certaines lignes du dataset apparaissent plusieurs fois. Si c'est le cas, effectuez le nécessaire pour ne garder qu'une seule occurrence de chaque ligne et expliquez votre démarche (1.5 pt)

In []: Démarche : Le code `df.duplicated().sum()` compte le nombre de lignes qui sont des

In []: Si des duplicatas existaient, la démarche à suivre serait :

Explication de la démarche (si des duplicatas avaient été trouvés) :

La méthode `df.drop_duplicates()` est utilisée pour supprimer les lignes qui sont

L'argument `inplace=True` modifie directement le DataFrame `df`.

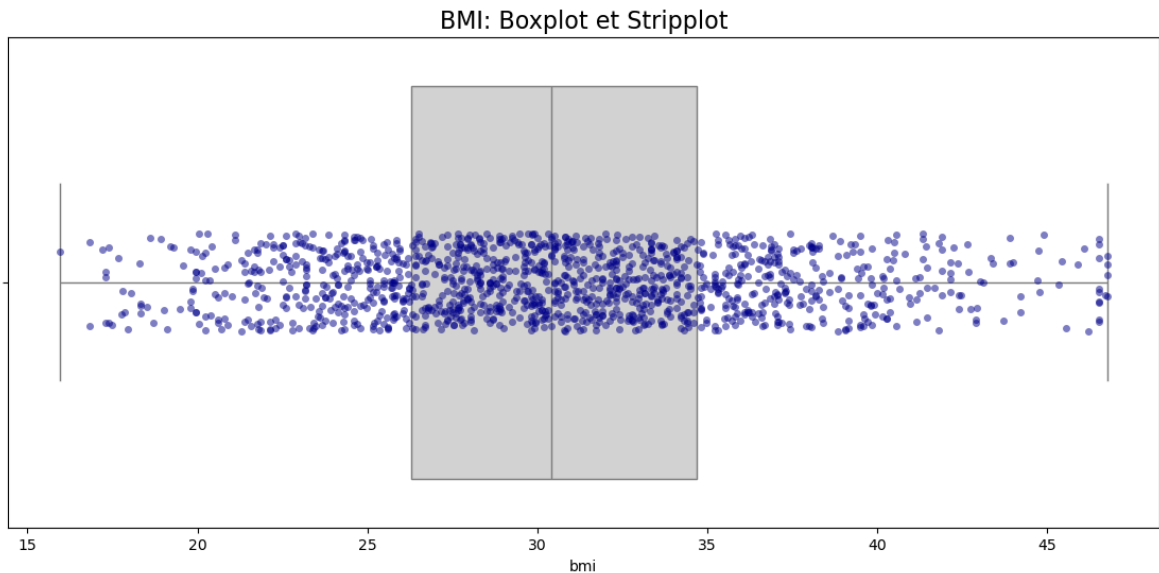
Cette étape est cruciale car les doublons introduisent un biais et peuvent entra

```
In [29]: print(f"Duplicates: {df.duplicated().sum()}")
```

Duplicates: 0

Q8: Compléter le code suivant pour afficher un boxplot et les points individuels de la variable `bmi` (1 pt)

```
In [13]: plt.figure(figsize=(14, 6))
sns.boxplot(x=df['bmi'], color='lightgray')
sns.stripplot(x=df['bmi'], color='darkblue', alpha=0.5)
plt.title('BMI: Boxplot et Stripplot', fontsize=16)
plt.show()
```



Q9: y-a-t-il des valeurs aberrantes dans la variable `bmi` (0.5 pt)

In []: Oui en se basant sur le graphique du boxplot :

Les valeurs aberrantes sont représentées par des points individuels au-delà des

Le boxplot de `bmi` montre généralement des points clairement séparés au-dessus de

Q10: Exécuter le code suivant et répondre aux questions suivantes : (1.5 pt)

```
In [22]: # 1. Define the Outliers (Global Logic)
Q1 = df['bmi'].quantile(0.25)
Q3 = df['bmi'].quantile(0.75)
IQR = Q3 - Q1
global_limit = Q3 + 1.5 * IQR
outlier_mask = df['bmi'] > global_limit
clean_max_values = df[~outlier_mask].groupby('sex')['bmi'].max()
print("Replacement Values (Max of normal data):")
print(clean_max_values)
df['bmi'] = np.where(
    outlier_mask,
    df['sex'].map(clean_max_values),
    df['bmi']
)
```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[22], line 7
      5 global_limit = Q3 + 1.5 * IQR
      6 outlier_mask = df['bmi'] > global_limit
----> 7 clean_max_values = df[~outlier_mask].groupby('sex')['bmi'].max()
      8 print("Replacement Values (Max of normal data):")
      9 print(clean_max_values)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:9183, in DataFrame.groupby(self, by, axis, level, as_index, sort, group_keys, observed, dropna)
    9180 if level is None and by is None:
    9181     raise TypeError("You have to supply one of 'by' and 'level'")
-> 9183 return DataFrameGroupBy(
    9184     obj=self,
    9185     keys=by,
    9186     axis=axis,
    9187     level=level,
    9188     as_index=as_index,
    9189     sort=sort,
    9190     group_keys=group_keys,
    9191     observed=observed,
    9192     dropna=dropna,
    9193 )

File ~\anaconda3\Lib\site-packages\pandas\core\groupby\groupby.py:1329, in GroupBy.__init__(self, obj, keys, axis, level, grouper, exclusions, selection, as_index, sort, group_keys, observed, dropna)
    1326 self.dropna = dropna
    1328 if grouper is None:
-> 1329     grouper, exclusions, obj = get_grouper(
    1330         obj,
    1331         keys,
    1332         axis=axis,
    1333         level=level,
    1334         sort=sort,
    1335         observed=False if observed is lib.no_default else observed,
    1336         dropna=self.dropna,
    1337     )
    1339 if observed is lib.no_default:
    1340     if any(ping._passed_categorical for ping in grouper.groupings):

File ~\anaconda3\Lib\site-packages\pandas\core\groupby\grouper.py:1043, in get_grouper(obj, key, axis, level, sort, observed, validate, dropna)
    1041     in_axis, level, gpr = False, gpr, None
    1042     else:
-> 1043         raise KeyError(gpr)
    1044 elif isinstance(gpr, Grouper) and gpr.key is not None:
    1045     # Add key to exclusions
    1046     exclusions.add(gpr.key)

KeyError: 'sex'

```

Q10

- (1 pt) C'est quoi l'objectif de ce code ? traiter les valeurs aberrantes
- (0.5 pt) À quoi servent Q1, Q3 et l'IQR dans cette logique ? Q1 (Premier Quartile) et Q3 (Troisième Quartile) : Ils délimitent les 50 % des données centrales. IQR

(InterQuartile Range) : Il mesure la dispersion des données centrales. Rôle dans la logique : Ces valeurs sont utilisées pour déterminer le seuil supérieur pour les valeurs aberrantes toutes les valeurs de bmi dépassant cette limite sont considérées comme des outliers à plafonner.

Q11: Compléter le code pour encoder les colonnes catégorielles avec One-Hot Encoding. (1.5 pt)

```
In [17]: from sklearn.preprocessing import OneHotEncoder

categorical_cols = ['sex', 'smoker', 'region']

encoder = OneHotEncoder(drop='first', sparse_output=False)

for col in categorical_cols:
    encoded = encoder.fit_transform(df[[col]])
    encoded_df = pd.DataFrame(encoded, columns=[f"{col}_{cat}" for cat in encode
df = pd.concat([df, encoded_df], axis=1)
df.drop(col, axis=1, inplace=True)
```

Q12: Séparer les variables explicatives (X) de la variable cible (y) (1 pt)

- X : contient toutes les features ou variables indépendantes utilisées pour prédire.
- y : contient la variable cible ou dépendante que l'on cherche à prédire.

```
In [51]: X = df.drop('charges', axis=1)

y = df['charges']
```

Q13: Compléter le code pour séparer les données en ensembles d'entraînement et de test, en utilisant 20 % des données pour le test et en fixant l'aléatoire à 42. (1 pt)

```
In [ ]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

Q14: Compléter le code pour standardiser les colonnes continues age , bmi et children . (1 pt)

```
In [ ]: scaler = StandardScaler()
cols_to_scale = ['age', 'bmi', 'children']

X_train[cols_to_scale] = scaler.fit_transform(X_train[cols_to_scale])

X_test[cols_to_scale] = scaler.transform(X_test[cols_to_scale])
```

Q15: Quel est l'impact de cette standardisation sur les valeurs des colonnes ? (1 pt)

```
In [ ]: L'impact de la standardisation est de transformer les valeurs de chaque colonne
```

Q16 : Créer et entraîner un modèle de régression linéaire sur les données d'entraînement. (1 pt)

```
In [23]: lr_model = LinearRegression()

lr_model.fit(X_train, y_train)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[23], line 3
      1 lr_model = LinearRegression()
----> 3 lr_model.fit(X_train, y_train)

NameError: name 'X_train' is not defined
```

Q17: Le modèle de régression linéaire `lr_model` contient plusieurs paramètres après l'entraînement (1 pt)

Exécutez les commandes suivantes et Expliquez clairement à quoi correspond **chaque paramètre affiché**. (1 pt)

```
In [61]: print('Intercept = ', lr_model.intercept_)

print('Coefficients : ', lr_model.coef_)
```

`lr_model.intercept_` (Ordonnée à l'origine ou Biais) :C'est la valeur prédite de la variable cible (charges) lorsque toutes les variables explicatives (features) sont égales à zéro.Dans l'équation de régression .`lr_model.coef_` (Coefficients de régression) :C'est un tableau de valeurs (une pour chaque variable explicative) représentant l'impact de chaque variable sur la variable cible.Chaque coefficient indique le changement attendu dans la variable cible (charges) pour une augmentation d'une unité dans la variable explicative correspondante \$, en maintenant toutes les autres variables constantes.

Q18: Calculer les prédictions `y_pred` du modèle puis évaluer ses performances en calculant :

- le `R2`
- le `MAE`
- le `RMSE` (1 pt)

```
In [ ]: y_pred = lr_model.predict(X_test)

r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Model Accuracy (R2 Score): {r2}")
print(f"Average Error (MAE): {mae}")
print(f"Root Mean Squared Error: {rmse}")
```

Q19 : Discuter la valeur de R2. (1.5 pt)

Une valeur élevée (par exemple, > 0.7) suggère que le modèle est pertinent et a une bonne capacité prédictive. Il faut noter que le R^2 seul ne garantit pas la validité du modèle (il peut y avoir surapprentissage, par exemple). C'est pourquoi il est essentiel de l'examiner en conjonction avec le MAE et le RMSE.

In []: